ST-LoRA: Space- and Time-based extension of LoRA

Anonymous ACL submission

Abstract

Parameter-Efficient Fine-Tuning (PEFT) methods have become leading frameworks to fine-tune Large Language Models on target task-specific datasets. Most of them are focused on idea to incorporate additional parameters directly by changing the decomposition method or addition of extra parameters by using designed adapters. In this paper, we are focused on idea to increase the capacity of the default LoRA approach by extending it through time (periodically merge the intermediate LoRA states into the original weights) and space (tackle with several copies of LoRA by using Mixture-of-Experts framework). The proposed solution allows to preserve the same inference time (thanks to the ability of merging the trained weights into the original ones). Extensive experiments demonstrated that the proposed method outperforms modern PEFT approaches on a wide range of datasets and model architectures under the new fine-tuning paradigm – eXtreme Fine-Tuning (XFT).

1 Introduction

Rapid scaling model sizes (e.g. GPT4 (OpenAI, 2023) is about 1.8 trillion parameters and it is 10 times more than its predecessor, GPT3 (Brown et al., 2020)) has allowed the Large Language Models (LLMs) to tackle with challenging human problems, like writing poems (Sawicki et al., 2023), solving math word tasks (Zhou et al., 2023), helping in medicine (Nori et al., 2023). The cost of mentioned above success is a dramatic growth of the time and the number of computation resources during the training phase (e.g. LLaMA-2-70B (Touvron et al., 2023) requires 1,720,320 GPU hours and up to 2000 GPU devices).

Unfortunately, such requirement to use a huge number of computation resources for training prevents a wider range of researchers to tackle with LLM challenges. One of possible solutions is to carry out fine-tuning on target dataset of already trained LLM (pre-training (He et al., 2018)). It reduces the training time but still requires almost the same number of computation devices. Recently, it was proposed to replace full fine-tuning onto its efficient variant: prompt-/prefix-tuning (Petrov et al., 2023; Wang et al., 2023b), training adapters (Houlsby et al., 2019), sparse training (Sung et al., 2021) and weight decomposition (Hu et al., 2021; Mahabadi et al., 2021).

The most famous representative of the methods from the last one group is LoRA (Hu et al., 2021) approach. A little while back, it was designed dozens modifications to enhance a quality and an efficiency (e.g. GLoRA (Chavan et al., 2023), AdaLoRA (Zhang et al., 2023c), LoHA (Hyeon-Woo et al., 2023)) but there is still a gap between full and efficient fine-tuning.

In our opinion, the gap in an accuracy is related to the limited capacity of existed PEFT approaches – all of them based on the idea that the delta for updating the original model weights is originated from a low-dimensional subspace (Aghajanyan et al., 2020). On practice, that rule works well enough for small and middle size datasets (in comparison to the model size) but gets stuck with huge ones. So, we need at some extension to the base approaches for efficient incorporating the additional fine-tuning space into the existing weight-decomposition methods.

Later, researchers have made several attempts to combine successful in LLMs framework, Mixture-of-Experts (MoE) (Fedus et al., 2022), with LoRA. MoELoRA (Liu et al., 2023a) proposes to map experts according to the domain, MoLoRA (Zadouri et al., 2023) implements MoE approach directly by incorporating the gating function, SiRA(Zhu et al., 2023) improves the accuracy by sparsifying the gating function, AdaMix (Wang et al., 2022) routes all tokens through the randomly selected expert. In this paper, we follow the same concept to extend the adapter capacity by integration with MoE

framework but do it in a different way to preserve the ability for merging the learnt weight delta to the original model weights for the efficient inference.

Another direction of extending the LoRA approach is based on the idea to merge the weight delta periodically, like ReLoRA (Lialin et al., 2023) (it merges a weight delta several times through the training) and DeltaLoRA (Zi et al., 2023) (runs merging on each iteration). We found that the last one technique is more accurate and incorporated it into the proposed solution.

Our contribution

- Proposed new extension of LoRA approach by using MoE framework to tackle with limited capacity issue of existing PEFT methods. It has a zero-latency during an inference due to the ability to merge a weight delta into the original weights.
- Augmented the method by enabling DeltaLoRA framework to get the better accuracy while preserving the desired zero-latency property.
- Performed an extensive validation to prove the effectiveness of the proposed solution in comparison to the other ones.

2 Related work

2.1 LLMs

Modern language model architectures become bigger to capture more complex knowledge and be able to perform on par with human in different challenging tasks. Due to using Self Attention (SA) (Vaswani et al., 2023) as core framework to process input sequences the LLM architectures suffers from an inefficiency and requires a huge amount of computations for training.

On the way to solve the mentioned problem, it was proposed several architecture-based approaches, like reducing the length of an attention (named Sliding Window Attention, SWA (Beltagy et al., 2020)), reducing the number of independent groups in a query (Grouped Query Attention, GQA (Ainslie et al., 2023)) and enabling a rolling buffer cache (RBC) (Jiang et al., 2023). The different direction is done in Flash Attention (FA) (Dao et al., 2022) framework which has performed low-level optimization of SA.

Woefully, the aforementioned approaches do not improve the situation dramatically and fine-tuning the biggest examples of LLMs still requires huge computation resources.

2.2 PEFT approaches

Establishing adapters (Houlsby et al., 2019) for NLP problem opened the direction of Parameter-Efficient Fine-Tuning (PEFT). The next attempt to adapt a pre-trained LLM model for a downstream task is represented by prompt (Lester et al., 2021) and prefix (Li and Liang, 2021) tuning methods and some modifications of them (P-Tuning (Liu et al., 2023b, 2022c), FL-Tuning (Liu et al., 2022b), MPT (Wang et al., 2023d)). Another chapter has been opened by introducing the low-rank matrix decomposition method, LoRA (Hu et al., 2021). It has several modifications, each of which changes some property: the type of matrix decomposition by using Kronecker product (LoHA (Hyeon-Woo et al., 2023) and KronA (Edalati et al., 2022)), the subset of model parameters for applying to (GLoRA (Chavan et al., 2023)), fine-tunes vector-based scales for random matrices (VeRA (Kopiczko et al., 2023)), adds the orthogonalization loss (O-LoRA (Wang et al., 2023a)). One more group of methods is based around the idea to quantize the original model weights to reduce the memory consumption: QLoRA (Dettmers et al., 2023), LoftQ (Li et al., 2023), LQ-LoRA (Guo et al., 2023). We follow the original idea of LoRA approach but apply the adapter for all linear modules, like it is done in QLoRA (note that we do not use any quantization).

Aiming to automatically pick out the rank for a low-rank decomposition it was proposed to minimize the number of non-zero pseudo-singular values (AdaLoRA (Zhang et al., 2023c)), train LoRA for all ranks simultaneously (DyLoRA (Valipour et al., 2023)), incrementally allocate parameters (IncreLoRA (Zhang et al., 2023a)), perform low-rank optimization steps (InRank (Zhao et al., 2023)). Our experiments showed that the mentioned above methods are not efficient in real scenarios (like text summarization and instruction tuning) and do not overcome the default LoRA in most of cases.

Different way to enhance the LoRA framework focuses on the idea to periodically merge the leant by LoRA adapter weights into the appropriate model weights. It allows to unchain the capacity of an original adapter for new knowledges from a dataset. The difference in existed methods is the frequency to apply merging: ReLoRA (Lialin et al., 2023) carry out merging several times during the

whole training but DeltaLoRA (Zi et al., 2023) do it on each iteration. According to our experiments, DeltaLoRA demonstrates consistent improvement over the LoRA baseline, so we use it in our paper too.

2.3 Model averaging

In light of the challenge for fine-tuning the same LLM for different tasks (in most difficult scenario the tasks are from different domains) researchers has proposed several solutions. LoRAHub (Huang et al., 2023) introduces the method to learn the weights for averaging the independent LoRA adapters to perform on some target dataset without fine-tuning the whole model on it, LLLA (Yang et al., 2023) uses a more complex bayesian low-rank adaptation. Another group of methods serves the independent LoRA weights without merging but providing efficient routing mechanism to preserve the target memory consumption: Punica (Chen et al., 2023), S-LoRA (Sheng et al., 2023), DeltaZip (Yao and Klimovic, 2023).

Similar to the mentioned above papers, we follow the idea to serve several LoRA copies but different from them we do it during the training stage only. Also we carry out weighted averaging of LoRA copies but individual weights are different on each training iteration.

2.4 Mixture-of-Experts (MoE)

Significant leap in a quality of LLM has been made by incorporating Mixture-of-Experts (MoE) (Fedus et al., 2022) framework to dramatically increase the number of model parameters while preserving almost the same computation budget. That suggestion has found the application in PEFT problem – MoELoRA (Liu et al., 2023a) routes samples from different domains through independent LoRA copies according to the predefined "domain-copy" mapping, MoLoRA (Zadouri et al., 2023) implements a learnable router for the same purposes, LoRA-TLE (Belofsky, 2023) applies "token-expert" mapping. SiRA (Zhu et al., 2023) enhances the accuracy of MoLoRA by replacing the dense routing mechanism onto the sparse one.

The main drawback of the aforementioned methods is inability to merge the routing module into the original weights. Opposite to that MultiLoRA (Wang et al., 2023c) propose to learn weights for each expert and carry out weighted averaging instead of routing by independent module. It allows to preserve merging ability. We follow the same idea but instead of direct learning the weights for averaging we estimate them by some special procedure. One more approach, AdaMix (Wang et al., 2022), overcomes the routing problem by randomly selecting a single expert for the whole batch of tokens. According to experiments, AdaMix works well enough but it can be enhanced by fusing with MultiLoRA, as it's done in our paper.

3 Background

In this chapter we will give brief overview of the related methods. As it was said before the proposed method is an extension of LoRA approach in some way. According to our experiments the main drawback of LoRA method is inability to fit huge datasets (close to full fine-tuning on chat / instructions datasets, e.g. fine-tuning GPT-3 (Brown et al., 2020) to get ChatGPT (OpenAI, 2022) model). Simple increasing the rank for low-rank decomposition fails for relatively small values of a rank – usually the quality stops enhancing after reaching the rank of 64 (Kalajdzievski, 2023). It means, that from the one hand we do not have enough capacity to fit the target dataset and should find a way to extend the LoRA while preserving the efficiency. On the other hand, training fails to reuse an extra capacity of LoRA with high rank due to dominating the main component in parameter updating (Zeng and Lee, 2023; Nikdan et al., 2024).

We propose to think about the possible extension of LoRA method in two orthogonal paradigms: space- and time-based extensions. For the first one we should manage several copies of LoRA adapter (experts) connected with the same weight and route the data flow through them during the training phase. Such idea is close to Mixture-of-Experts and AdaMix approaches and bellow we give the description of them. To implement the latest paradigm we should iteratively merge the current state of LoRA adapter into the original weights and reset the LoRA adapter after this thereby freeing the space for the new knowledge from a training dataset. That approach has been implemented in ReLoRA / DeltaLoRA methods and below we give the description of them too.

Generally speaking, we want to increase the total rank of low-rank weight decomposition but do it in different way to overcome the possible issues with training procedure. It means we aim to achieve the best quality while training extremely shortly our

target network. To do that we exploit the property of sum of matrices: $rank(\mathbf{X} + \mathbf{Y}) \leq rank(\mathbf{X}) + rank(\mathbf{Y})$. So, increasing the number of terms in the equation we also increase the total rank of a fused adapter.

3.1 Weight decomposition

For a given pre-trained model weight $\mathbf{W} \in \mathbb{R}^{m \times n}$ LoRA (Hu et al., 2021) learns an incremental update $\Delta\mathbf{W}$, where matrix $\Delta\mathbf{W}$ is decomposed into the matrix multiplication of two low-rank matrices $\mathbf{A} \in \mathbb{R}^{m \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times n}$:

$$y = \mathbf{W}x + \Delta\mathbf{W}x = \mathbf{W}x + \frac{\alpha}{r}\mathbf{A}\mathbf{B}x \quad (1)$$

where x is an input of linear layer and y is the output. The rank r is chosen significantly lower than dimensions of matrix $\mathbf{W}$: $r \ll min(m, n)$. To smoothly integrate the LoRA adapters into the existing network the matrices $\mathbf{A}$ and $\mathbf{B}$ are randomly initialized via Kaiming (He et al., 2015) and zero initializations respectively, thereby providing $\Delta\mathbf{W} = 0$ on the first step of training. Also LoRA uses hyper-parameter α to control the scale of updates. During the training phase a matrix $\mathbf{W}$ are frozen and only matrices $\mathbf{A}$ and $\mathbf{B}$ are updated through the back-propagation procedure.

3.2 Time-based extension

Instead of following the original LoRA paradigm to carry out merging the delta weights only once after finishing the training procedure it is also possible to update the original model weights $\mathbf{W}$ iteratively, like in DeltaLoRA (Zi et al., 2023):

$$\mathbf{W}_{t+1} = \mathbf{W}_t + \lambda\frac{\alpha}{r}(\mathbf{A}_{t+1}\mathbf{B}_{t+1} - \mathbf{A}_t\mathbf{B}_t) \quad (2)$$

where λ is hyper-parameter to control the trade-off between the update ration $\mathbf{AB}$ and original model weights $\mathbf{W}$, t and $t+1$ are two consecutive iterations of training.

Also there is similar approach ReLoRA (Lialin et al., 2023) which updates the original weights $\mathbf{W}$ too but several times only through the whole training and by merging the learnt LoRA delta $\mathbf{AB}$ directly into $\mathbf{W}$. On practice, it works similar to default LoRA and do not improve it. So, we use DeltaLoRA for all future experiments due to strong improvement in a quality in comparison to LoRA.

3.3 Space-based extension

The idea to extend the default set of parameters by an extra space and some routing mechanism over it is originated from Mixture-of-Experts paper (Fedus et al., 2022). Here we review its application for LoRA method, MoLoRA (Zadouri et al., 2023). Assume, we have k independent copies of LoRA adapter: $L = \{\Delta\mathbf{W}^{(i)} = \mathbf{A}^{(i)}\mathbf{B}^{(i)}, i = \overline{1,k}\}$. According to the MoE framework we have a routing module $R(x) = softmax(\mathbf{W}_g^T x)$, where x is a module input. So the output of MoLoRA is next:

$$y = \mathbf{W}x + \sum_{i=1}^{k} R(x)^{(i)}\Delta\mathbf{W}^{(i)}x \quad (3)$$

The main drawback of the mentioned above method is inability to fuse $\Delta\mathbf{W}^i$ into $\mathbf{W}$ due to the router $R(x)$, which depends on the module input.

The opposite solution is proposed by AdaMix (Wang et al., 2022) approach – it has stochastic routing which randomly selects the pair of LoRA matrices $\Delta\mathbf{W}^{(j)}$ from L and then apply it for the all samples in a batch:

$$y = \mathbf{W}x + \Delta\mathbf{W}^{(j)}x \quad (4)$$

On practice for the real use-case scenarios the AdaMix method do not improve the LoRA baseline but preserves the training and inference efficiency.

4 Method

As it was mentioned early the proposed solution is based on MoE approach but without an explicit router module. To preserve the ability for fusing the copies of LoRA into the original model weights we are limited to find a solution of the next form:

$$y = \mathbf{W}x + \sum_{i=1}^{k} \omega_i\Delta\mathbf{W}^{(i)}x \quad (5)$$

where $\omega_i \geq 0, i = \overline{1,k}$ and $\sum_{i=1}^{k}\omega_i = 1$ are weights of individual LoRA copies (experts). The similar idea is proposed in MultiLoRA (Wang et al., 2023c), where expert weights ω_i are learnable parameters but we found such idea is suboptimal on the practice due to a probability to collapse a weight distribution to a simple uniform.

Instead of it we implement the routing mechanism implicitly. First of all, lets define the "token-expert" mapping function: $M : \mathbb{N} \to \mathbb{N}$, which assign for each input token the "ID" of one of a

possible expert ($\Delta\mathbf{W}^{(i)}$). Here we implemented random uniform mapping. After mapping we can calculate the counts and frequencies of each expert for the current tokens in an input x: $S(x) = (s_1(x), s_2(x), ..., s_k(x))$. Then we are able to estimate expert weights on current iteration by the following equation:

$$\Omega(x) = (\omega_1(x), \omega_2(x), ..., \omega_k(x)) = softmax(\beta S(x)) \tag{6}$$

Here β is a scale to control the sharpness of expert weights distribution. In our experiments that hyper-parameter is set to 10. The aforementioned formulation is used during training only and to accumulate the expert weights Ω for the inference stage we track the exponential average of them (the momentum value is 0.9).

Finally, we can combine the introduced space-based extension of LoRA with time-based formulation, DeltaLoRA:

$$\mathbf{W}_{t+1} = \mathbf{W}_t + \lambda \frac{\alpha}{r} \sum_{i=1}^{k} \omega_i(x)(\mathbf{A}_{t+1}^{(i)}\mathbf{B}_{t+1}^{(i)} - \mathbf{A}_t^{(i)}\mathbf{B}_t^{(i)}) \tag{7}$$

Below you can see the full algorithm of the proposed Space- and Time-based extension of LoRA approach (named ST-LoRA):

Algorithm 1 ST-LoRA

Require: Total training iterations T_{max}; rank r; scale factor α; start steps T_{init}; update ratio λ; momentum μ; vocabulary size V; number of experts k; sharpness ratio β.

Ensure: $\mathbf{A}_0^{(i)}$ by Kaiming uniform, $\mathbf{B}_0^{(i)} = 0$, $\mathbf{W}_0$ with pre-trained weights, mapping $M : \{1, ..., V\} \to \{1, ..., k\}$ by random uniform.

for $t = \overline{0, T_{max} - 1}$ **do**
  Sample a batch x
  Map all input tokens in the batch into expert IDs
  Count the number of tokens for each expert $(n_1, n_2, ..., n_k)$
  $s_i = n_i / \sum_j n_j$ ▷ Calculate expert frequencies
  $\omega_i = e^{\beta s_i} / \sum_j e^{\beta s_j}$ ▷ Estimate expert weights
  if t > 0 **then**
    $\overline{\omega_i} = m\overline{\omega_i} + (1 - m)\omega_i$ ▷ Update final expert weights
  else
    $\overline{\omega_i} = \omega_i$
  end if
  $\Delta\mathbf{W}_t = \lambda \frac{\alpha}{r} \sum_{i=1}^{k} \omega_i \mathbf{A}_t^{(i)} \mathbf{B}_t^{(i)}$
  $y = \mathbf{W}_t x + \Delta\mathbf{W}_t x$ ▷ Calculate module output
  $\mathbf{A}_{t+1} = \mathbf{A}_t + \Delta\mathbf{A}_t$ ▷ Gradient update
  $\mathbf{B}_{t+1} = \mathbf{B}_t + \Delta\mathbf{B}_t$ ▷ Gradient update
  if $t \geq T_{init}$ **then**
    $\mathbf{W}_{t+1} = \mathbf{W}_t + \lambda \frac{\alpha}{r} \sum_{i=1}^{k} \omega_i (\mathbf{A}_{t+1}^{(i)}\mathbf{B}_{t+1}^{(i)} - \mathbf{A}_t^{(i)}\mathbf{B}_t^{(i)})$
  end if
end for
Fusing: $\Delta\mathbf{W}_{T_{max}} = \lambda \frac{\alpha}{r} \sum_{i=1}^{k} \overline{\omega_i} \mathbf{A}_{T_{max}}^{(i)} \mathbf{B}_{T_{max}}^{(i)}$
Output: The fine-tuned parameters $(\mathbf{W}_{T_{max}}, \Delta\mathbf{W}_{T_{max}})$

5 Experiments

To verify the effectiveness of the proposed method we have evaluated it on a wide range of LLM architectures and datasets. It worth to say, that we are limited in choice of big enough datasets – our approach is based on the idea to significantly increase the number of parameters in some efficient way which faces the problem of possible over-fitting in case of datasets with relatively small number of records.

It was selected the problem of a natural language generation which mostly reflects the modern requests for LLMs from users. The first challenge is the text summarization problem which is represented by the Extreme Summarization (XSum) (Narayan et al., 2018) dataset. To solve the mentioned dataset the network should generate the abstractive summarization for each input document. The dataset consists of 226,711 news articles accompanied with a one-sentence summary. The articles are collected from BBC articles (from 2010 to 2017) and cover a wide variety of domains (e.g. news, politics, sports, weather, business, technology, science, health, family, education, entertainment and arts). We follow the official split of dataset on train and val subparts. Another challenging dataset is GPT4All (Anand et al., 2023) which is suitable for training an assistant-style chat-bot. The dataset consists of 809,000 "prompt-response" pairs of assistant interactions including word problems, multi-turn dialog, code, poems, songs, and stories.

We evaluate the proposed method, ST-LoRA,

Model name	Warm-up	η	α	β	λ	T_{init}
T5-Small	500	4e-4	16	10	4	1000
BART-Large	500	4e-4	16	10	4	1000
OpenLLaMA-3B	500	4e-7	16	10	8	250

Table 1: Hyper-parameter selection

with T5-Small (Raffel et al., 2023), BART-Large (Lewis et al., 2019) and OpenLLaMA-3B (Geng and Liu, 2023). Specifically, we train T5 and BART models on XSUM, and OpenLLaMA on GPT4All datasets. We choose encoder-decoder (T5 and BART) and decoder-only (Open-LLaMA) architectures to verify the scalability of the proposed approach. For validation purposes we use ROUGE (Lin, 2004) (XSUM dataset) and Perplexity (Chen et al., 2001) (GPT4All dataset) metrics.

5.1 Training

Note, we follow the alternative paradigm of PEFT and focusing on eXtreme Fine-Tuning problem (XFT) – we are trying to carry out fine-tuning as fast as possible. To do it we limit the total number of epochs (usually to 2) and focusing on methods providing fastest converge to the best quality. So, for comparison all other approaches has been reimplemented and retrained by us under the XFT formulation of PEFT problem.

To softly connect the PEFT adapters with existing model weights we start training with linear warm-up (Vaswani et al., 2023) following the default training with a linearly decayed learning rate. We apply PEFT methods (including the proposed one) for all linear weights in LLM except the embedding layer. By default all experiments uses 4 copies (experts) of LoRA / DeltaLoRA. Each copy is initialized independently according to the original practice from LoRA paper: matrix $\mathbf{A}$ is initialized by Kaiming uniform distribution and $\mathbf{B}$ is zero. In Table 1 you can find the values for hyper-parameters which have been used during experiments. All experiments have been done in PyTorch (Paszke et al., 2017) framework.

5.2 Text summarization task

For the text summarization problem it is selected two network architectures: T5-Small and BART-Large. In addition to the proposed solution it has been done a lot of different experiments of other PEFT approaches under the XFT formulation of problem. All results are presented in Table 2 for T5-Small and in Table 3 for BART-Large models.

From the Table 2 we can see that modern PEFT approaches do not work better then LoRA / GLoRA approaches. We consider GLoRA as extension of LoRA in terms of place for applying LoRA, so it is expected for improving the LoRA baseline by increasing the number of tunable parameters. But that rule is not general, e.g. LoHA doubles number of parameters but the final metric is worse than default LoRA. One more comparison is based on increasing the rank for the original LoRA to demonstrate the limited ability of that approach to increase the final metric by simple increasing the number of parameters (GLoRA increases the number of parameters too but in a different way).

The proposed solution demonstrates the strong improvement in case of space-only and space-plus-time extensions of LoRA. That result is preserved for both network architectures and has the large margin in case of T5-Small model. To demonstrate the scalability of the approach it has been done some experiments with increasing the number of experts (LoRA copies) – see the Table 4. As you can see the method (space-only part of method is used for experiment) allows to get larger the quality after increasing the number of copies. On the way to preserve the efficiency we do not see the reasons to use more than 4 experts.

Unfortunately, there is still a gap between full fine-tuning and any PEFT approach under XFT scenario. The main reason for that is a difficulty of the optimizing manifold which may become more sharp thereby preventing the successful optimization by first-oder methods. But the proposed solution improves the situation and allows to fit that gap with the acceptable value.

5.3 Instruction task

The next challenging dataset for validating the approach is the instruction dataset, GPT4All. We run it with OpenLLaMA-3B network architecture and the results are showed in Table 5. Here the situation is the same as before – new method outperforms the LoRA baseline with valuable margin. This achievement proves that the direction to extend the LoRA capacity by increasing the number of its copies through time and space works for the big network architectures too. It moves us to the idea that the bottleneck for efficient fine-tuning is the capacity of single adapter. Note, such behavior emerges for huge and difficult datasets, like XSUM and GPT4All – for the rest cases the default LoRA is mostly enough. Anyway, the ST-LoRA works

Method	Parameters ratio, %	ROUGE 1	2	L
Full fine-tuning	100	34.57	12.01	27.21
Zero-shot	0	19.28	2.79	13.70
LoRA rank-8 (Hu et al., 2021)	1.76	30.85	8.90	23.84
LoRA rank-16	3.45	30.89	8.94	23.84
LoRA rank-32	6.67	30.77	8.88	23.80
LoRA rank-64	12.51	30.77	8.85	23.79
IA3 (Liu et al., 2022a)	0.14	27.54	6.74	20.99
AdaLoRA (Zhang et al., 2023c)	1.76	29.80	8.19	22.98
DyLoRA (Valipour et al., 2023)	1.76	29.46	7.95	22.61
IncreLoRA (Zhang et al., 2023a)	2.13	29.40	7.96	22.58
FacT (Jie and Deng, 2023)	0.27	29.16	7.77	22.37
LoHA (Hyeon-Woo et al., 2023)	3.45	30.06	8.30	23.16
LoRA-FA (Zhang et al., 2023b)	0.88	30.90	9.03	23.94
LoHA + FA †	1.72	31.11	9.20	24.14
ROSA (Hameed and Rabusseau, 2023)	1.75	31.16	9.13	24.08
GLoRA (Chavan et al., 2023)	4.48	31.19	9.18	24.13
ReLoRA (Lialin et al., 2023)	1.76	31.18	9.12	24.10
DeltaLoRA (Zi et al., 2023)	1.76	31.35	9.32	24.31
AdaMix (Wang et al., 2022)	6.67	30.14	8.45	23.26
Time †	1.76	31.53	9.44	24.43
Space †	6.67	30.91	8.95	23.90
Space + Time †	6.67	31.70	9.54	24.58

Table 2: Results with T5-Small model on XSUM dataset. The custom modification and the proposed solution are marked by †.

Method	Parameters ratio, %	ROUGE 1	2	L
Full fine-tuning	100	44.09	20.81	35.80
Zero-shot	0	17.29	3.23	10.99
LoRA (Hu et al., 2021)	1.05	43.10	19.69	34.79
GLoRA (Chavan et al., 2023)	2.71	43.15	19.75	34.65
LoHA + FA †	1.05	42.50	19.25	34.29
LoRA-FA (Zhang et al., 2023b)	0.52	42.54	19.16	34.15
FacT (Jie and Deng, 2023)	0.25	42.04	18.60	33.67
DeltaLoRA (Zi et al., 2023)	1.05	43.27	19.92	34.95
Space †	4.08	43.41	19.97	35.12
Space + Time †	4.08	43.70	20.36	35.39

Table 3: Results with BART-Large model on XSUM dataset. The proposed solution is marked by †.

Method	Parameters ratio, %	ROUGE 1	2	L
Full fine-tuning	100	34.57	12.01	27.21
4 experts	6.67	30.91	8.95	23.90
8 experts	12.51	30.89	8.90	23.82
16 experts	22.24	30.98	8.98	23.95

Table 4: Scalability of the proposed approach with different number of experts with T5-Small model on XSUM dataset.

Method	Parameters ratio, %	Perplexity
LoRA (Hu et al., 2021)	0.37	284.78
Space + Time †	1.46	227.75

Table 5: Results with OpenLLaMA-3B model on GPT4All dataset. The proposed solution is marked by †.

even for that scenario.

6 Conclusion

By changing the paradigm of extension the LoRA method into two orthogonal directions (space and time) we have designed the new approach on top of existing best working methods. That solution allows to overcome the limitation of original LoRA to fit huge datasets and continue to improve the final model quality by increasing the total rank of a weight decomposition. The approach has been confirmed on a wide range of network architectures and datasets and demonstrated the strong improvement over the LoRA framework and its previous enhancements. For the future work the proposed solution should be optimized in terms of memory consumption for the space-based part of approach.

7 Limitations

Despite the demonstrated improvement over the LoRA baseline the proposed solution has some drawbacks. The space-based part of solution requires the additional memory on device which can be expensive in case of big LLMs. There are several approaches to tackle it: offloading on CPU some tensors, gradient check-pointing, quantization. For the last one method there is an issue with applying DeltaLoRA over the quantized weights of a source model – we need at some buffer to accumulate the delta for the original weights before merging and quantizing with them. It also increases the total memory consumption but still is more efficient then full fine-tuning. Fortunately, the described method preserve the zero-latency advantage of the original LoRA approach.

References

Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. 2020. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *Preprint*, arXiv:2012.13255.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *Preprint*, arXiv:2305.13245.

Yuvanesh Anand, Zach Nussbaum, Adam Treat, Aaron Miller, Richard Guo, Ben Schmidt, GPT4All Community, Brandon Duderstadt, and Andriy Mulyar. 2023. Gpt4all: An ecosystem of open source compressed language models. *Preprint*, arXiv:2311.04931.

Joshua Belofsky. 2023. Token-level adaptation of lora adapters for downstream task generalization. *Preprint*, arXiv:2311.10847.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *Preprint*, arXiv:2004.05150.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. *Preprint*, arXiv:2005.14165.

Arnav Chavan, Zhuang Liu, Deepak Gupta, Eric Xing, and Zhiqiang Shen. 2023. One-for-all: Generalized lora for parameter-efficient fine-tuning. *Preprint*, arXiv:2306.07967.

Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. Punica: Multi-tenant lora serving. *Preprint*, arXiv:2310.18547.

Stanley Chen, Douglas Beeferman, and Ronald Rosenfeld. 2001. Evaluation metrics for language models.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Preprint*, arXiv:2205.14135.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Preprint*, arXiv:2305.14314.

Ali Edalati, Marzieh Tahaei, Ivan Kobyzev, Vahid Partovi Nia, James J. Clark, and Mehdi Rezagholizadeh. 2022. Krona: Parameter efficient tuning with kronecker adapter. *Preprint*, arXiv:2212.10650.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Preprint*, arXiv:2101.03961.

Xinyang Geng and Hao Liu. 2023. Openllama: An open reproduction of llama.

Han Guo, Philip Greengard, Eric P. Xing, and Yoon Kim. 2023. Lq-lora: Low-rank plus quantized matrix decomposition for efficient language model finetuning. *Preprint*, arXiv:2311.12023.

M. G. A. Hameed and G. Rabusseau. 2023. Rosa: Random orthogonal subspace adaptation.

Kaiming He, Ross Girshick, and Piotr Dollár. 2018. Rethinking imagenet pre-training. *Preprint*, arXiv:1811.08883.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *Preprint*, arXiv:1502.01852.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. *Preprint*, arXiv:1902.00751.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *Preprint*, arXiv:2106.09685.

Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. 2023. Lorahub: Efficient cross-task generalization via dynamic lora composition. *Preprint*, arXiv:2307.13269.

Nam Hyeon-Woo, Moon Ye-Bin, and Tae-Hyun Oh. 2023. Fedpara: Low-rank hadamard product for communication-efficient federated learning. *Preprint*, arXiv:2108.06098.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *Preprint*, arXiv:2310.06825.

Shibo Jie and Zhi-Hong Deng. 2023. Fact: Factor-tuning for lightweight adaptation on vision transformer. *Preprint*, arXiv:2212.03145.

Damjan Kalajdzievski. 2023. A rank stabilization scaling factor for fine-tuning with lora. *Preprint*, arXiv:2312.03732.

Dawid Jan Kopiczko, Tijmen Blankevoort, and Yuki Markus Asano. 2023. Vera: Vector-based random matrix adaptation. *Preprint*, arXiv:2310.11454.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *Preprint*, arXiv:2104.08691.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *Preprint*, arXiv:1910.13461.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *Preprint*, arXiv:2101.00190.

Yixiao Li, Yifan Yu, Chen Liang, Pengcheng He, Nikos Karampatziakis, Weizhu Chen, and Tuo Zhao. 2023. Loftq: Lora-fine-tuning-aware quantization for large language models. *Preprint*, arXiv:2310.08659.

Vladislav Lialin, Namrata Shivagunde, Sherin Muckatira, and Anna Rumshisky. 2023. Relora: High-rank training through low-rank updates. *Preprint*, arXiv:2307.05695.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries.

Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. 2022a. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *Preprint*, arXiv:2205.05638.

Jingping Liu, Yuqiu Song, Kui Xue, Hongli Sun, Chao Wang, Lihan Chen, Haiyun Jiang, Jiaqing Liang, and Tong Ruan. 2022b. Fl-tuning: Layer tuning for feed-forward network in transformer. *Preprint*, arXiv:2206.15312.

Qidong Liu, Xian Wu, Xiangyu Zhao, Yuanshao Zhu, Derong Xu, Feng Tian, and Yefeng Zheng. 2023a. Moelora: An moe-based parameter efficient fine-tuning method for multi-task medical applications. *Preprint*, arXiv:2310.18339.

Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2022c. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *Preprint*, arXiv:2110.07602.

Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. 2023b. Gpt understands, too. *Preprint*, arXiv:2103.10385.

Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. 2021. Compacter: Efficient low-rank hypercomplex adapter layers. *Preprint*, arXiv:2106.04647.

Shashi Narayan, Shay B. Cohen, and Mirella Lapata. 2018. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *Preprint*, arXiv:1808.08745.

Mahdi Nikdan, Soroush Tabesh, and Dan Alistarh. 2024. Rosa: Accurate parameter-efficient fine-tuning via robust adaptation. *Preprint*, arXiv:2401.04679.

Harsha Nori, Nicholas King, Scott Mayer McKinney, Dean Carignan, and Eric Horvitz. 2023. Capabilities of gpt-4 on medical challenge problems. *Preprint*, arXiv:2303.13375.

OpenAI. 2022. Introducing chatgpt.

OpenAI. 2023. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.

Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. 2017. Automatic differentiation in pytorch.

Aleksandar Petrov, Philip H. S. Torr, and Adel Bibi. 2023. When do prompting and prefix-tuning work? a theory of capabilities and limitations. *Preprint*, arXiv:2310.19698.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2023. Exploring the limits of transfer learning with a unified text-to-text transformer. *Preprint*, arXiv:1910.10683.

Piotr Sawicki, Marek Grzes, Fabricio Goes, Dan Brown, Max Peeperkorn, and Aisha Khatun. 2023. Bits of grass: Does gpt already know how to write like whitman? *Preprint*, arXiv:2305.11064.

Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2023. S-lora: Serving thousands of concurrent lora adapters. *Preprint*, arXiv:2311.03285.

Yi-Lin Sung, Varun Nair, and Colin Raffel. 2021. Training neural networks with fixed sparse masks. *Preprint*, arXiv:2111.09839.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. *Preprint*, arXiv:2307.09288.

Mojtaba Valipour, Mehdi Rezagholizadeh, Ivan Kobyzev, and Ali Ghodsi. 2023. Dylora: Parameter efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. *Preprint*, arXiv:2210.07558.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention is all you need. *Preprint*, arXiv:1706.03762.

Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. 2023a. Orthogonal subspace learning for language model continual learning. *Preprint*, arXiv:2310.14152.

Yaqing Wang, Sahaj Agarwal, Subhabrata Mukherjee, Xiaodong Liu, Jing Gao, Ahmed Hassan Awadallah, and Jianfeng Gao. 2022. Adamix: Mixture-of-adaptations for parameter-efficient model tuning. *Preprint*, arXiv:2205.12410.

Yihan Wang, Jatin Chauhan, Wei Wang, and Cho-Jui Hsieh. 2023b. Universality and limitations of prompt tuning. *Preprint*, arXiv:2305.18787.

Yiming Wang, Yu Lin, Xiaodong Zeng, and Guannan Zhang. 2023c. Multilora: Democratizing lora for better multi-task learning. *Preprint*, arXiv:2311.11501.

Zhen Wang, Rameswar Panda, Leonid Karlinsky, Rogerio Feris, Huan Sun, and Yoon Kim. 2023d. Multitask prompt tuning enables parameter-efficient transfer learning. *Preprint*, arXiv:2303.02861.

Adam X. Yang, Maxime Robeyns, Xi Wang, and Laurence Aitchison. 2023. Bayesian low-rank adaptation for large language models. *Preprint*, arXiv:2308.13111.

Xiaozhe Yao and Ana Klimovic. 2023. Deltazip: Multi-tenant language model serving via delta compression. *Preprint*, arXiv:2312.05215.

Ted Zadouri, Ahmet Üstün, Arash Ahmadian, Beyza Ermiş, Acyr Locatelli, and Sara Hooker. 2023. Pushing mixture of experts to the limit: Extremely parameter efficient moe for instruction tuning. *Preprint*, arXiv:2309.05444.

Yuchen Zeng and Kangwook Lee. 2023. The expressive power of low-rank adaptation. *Preprint*, arXiv:2310.17513.

Feiyu Zhang, Liangzhi Li, Junhao Chen, Zhouqiang Jiang, Bowen Wang, and Yiming Qian. 2023a. Increlora: Incremental parameter allocation method for parameter-efficient fine-tuning. *Preprint*, arXiv:2308.12043.

Longteng Zhang, Lin Zhang, Shaohuai Shi, Xiaowen Chu, and Bo Li. 2023b. Lora-fa: Memory-efficient low-rank adaptation for large language models fine-tuning. *Preprint*, arXiv:2308.03303.

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023c. Adaptive budget allocation for parameter-efficient fine-tuning. *Preprint*, arXiv:2303.10512.

Jiawei Zhao, Yifei Zhang, Beidi Chen, Florian Schäfer, and Anima Anandkumar. 2023. Inrank: Incremental low-rank learning. *Preprint*, arXiv:2306.11250.

Aojun Zhou, Ke Wang, Zimu Lu, Weikang Shi, Sichun Luo, Zipeng Qin, Shaoqing Lu, Anya Jia, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2023. Solving challenging math word problems using gpt-4 code interpreter with code-based self-verification. *Preprint*, arXiv:2308.07921.

Yun Zhu, Nevan Wichers, Chu-Cheng Lin, Xinyi Wang, Tianlong Chen, Lei Shu, Han Lu, Canoee Liu, Liangchen Luo, Jindong Chen, and Lei Meng. 2023. Sira: Sparse mixture of low rank adaptation. *Preprint*, arXiv:2311.09179.

Bojia Zi, Xianbiao Qi, Lingzhi Wang, Jianan Wang, Kam-Fai Wong, and Lei Zhang. 2023. Delta-lora: Fine-tuning high-rank parameters with the delta of low-rank matrices. *Preprint*, arXiv:2309.02411.